UNIT I: Principles of Object Oriented Programming & Control Structures

Question Bank: Debugging & Pseudo-Code Formulation (Questions Only)

---

1. Syllabus & Topic Scope

- **Object Oriented Programming Paradigm**: Procedural vs. Object-Oriented paradigms, benefits and limitations.
- **Basic Concepts of OOP**: Objects, Classes, Data Abstraction, Encapsulation, Inheritance, Polymorphism, Dynamic Binding, Message Passing.
- **Object Oriented Languages & Applications**: Characteristics of OOP languages, real-world application domains.
- **Tokens, Keywords, Identifiers & Constants**: C++ tokens, naming rules, literals, `const` qualifier vs. `#define`.
- **Data Types, Type Compatibility & Variables**: Primitive, derived, and user-defined data types, dynamic initialization, reference variables.
- **Operators in C++**: Scope resolution operator (`::`), memory management operators (`new`, `delete`), member dereferencing, bitwise, logical, relational, and ternary operators.
- **Implicit Conversions & Type Casting**: Promotion rules, standard type casting, potential precision loss.
- **Operator Precedence & Associativity**: Evaluation order of compound expressions.
- **Control Structures**: Selection statements (`if`, `if-else`, nested `if`, `switch-case`), Iteration statements (`while`, `do-while`, `for`), Jump statements (`break`, `continue`, `goto`, `return`).

---

2. Assessment Instructions & Rules ("How to Do & What to Do")

A. Instructions for Debugging Questions

When answering debugging questions, you must follow a structured, step-by-step analytical approach:

1. Analyze the Problem Statement: Read the expected functionality versus what the code currently does.

2. Examine the Code: Inspect each line carefully for:

- **Syntax Errors**: Missing tokens, invalid keywords, incorrect punctuation, misplaced semicolons.
- **Semantic & Type Errors**: Type incompatibilities, illegal implicit conversions, invalid reference bindings, illegal constant mutations.
- **Precedence & Associativity Traps**: Ambiguous compound expressions, bitwise vs. equality precedence conflicts.
- **Control Flow Flaws**: Unintended loop bounds, unsigned integer underflow/overflow, missing `break` statements in `switch`, invalid case labels.

- **Memory Management**: Mismatches between allocation (`new` / `new[]`) and deallocation (`delete` / `delete[]`).

3. Your Response for Each Question Must Include:

- **Line Number & Error Classification**: State the exact line number and categorize the defect (e.g., *Compilation Error*, *Runtime Bug*, *Undefined Behavior*, or *Logic Defect*).
- **Root Cause Explanation**: Explain *why* the compiler rejects the line or why the CPU generates invalid results, referencing specific C++ rules.
- **Corrected Code**: Provide the complete, compilable replacement code snippet.
- **Dry-Run & Predicted Output**: Provide the exact output generated by the corrected program.

B. Instructions for Pseudo-Code Writing Questions

When formulating pseudo-code, adhere to standard algorithmic and OOP conventions:

1. Algorithmic Structure:

- Use uppercase keywords for control structures: `BEGIN`, `END`, `IF`, `THEN`, `ELSE`, `ENDIF`, `WHILE`, `DO`, `ENDWHILE`, `FOR`, `TO`, `STEP`, `ENDFOR`.
- For Object-Oriented designs, define clear structural blocks: `CLASS <ClassName>`, `PRIVATE:`, `PUBLIC:`, `CONSTRUCTOR`, `METHOD`, `ENDCLASS`.

2. Variable Clarity:

- Explicitly declare variable names and types (`INTEGER`, `REAL`, `BOOLEAN`, `STRING`, `ARRAY`).

3. Your Response for Each Question Must Include:

- **Design Overview**: Brief bullet-point explanation of your chosen algorithm and OOP abstractions.
- **Structured Pseudo-Code**: Complete, cleanly indented pseudo-code without shortcuts or language-specific idioms.
- **Trace Table / Dry Run**: A structured table verifying the execution of your pseudo-code against the provided sample input.

---

3. Section A: Debugging Questions (5 Problems)

Question 1: Operator Precedence Trap with Bitwise Operations & Implicit Promotion

Topic: Operators in C++, Operator Precedence, Implicit Conversion

Problem Statement

A student intends to write a utility routine that verifies whether a given positive 8-bit integer is **both an odd number and strictly greater than 10**. If both conditions are satisfied, it should display `"Valid Odd"`. In addition, the routine attempts to count down from 3 to 0 using an unsigned loop counter.

However, the program either fails to execute as intended or enters an infinite loop.

Buggy Code

```
#include <iostream>
```

```
int main() {
    unsigned char num = 15;

    // Line 7: Intended to check if num is odd AND greater than 10
    if (num & 1 == 1 && num > 10) {
        std::cout << "Valid Odd Number: " << (int)num << std::endl;
    } else {
        std::cout << "Condition Failed for: " << (int)num << std::endl;
    }

    // Line 14: Intended to decrement from 3 down to 0 inclusive
    for (unsigned char i = 3; i >= 0; --i) {
        std::cout << (int)i << " ";
        if (i == 0) break; // Added as a temporary patch
    }
    std::cout << std::endl;

    return 0;
}
```

Student Tasks

1. Identify the operator precedence conflict on **Line 7**. Explain how the compiler evaluates the expression `num & 1 == 1` and why this causes logical bugs for various inputs.

2. Examine the `for` loop on **Line 14**. Explain why removing the `break` statement results in an infinite loop due to the properties of `unsigned char`.

3. Provide the corrected, robust C++ code snippet.

4. Show the dry run and exact console output of the corrected code.

Question 2: Scope Resolution & Variable Shadowing Conflicts

Topic: Variables, Scope Resolution Operator (`::`), Constants

Problem Statement

An e-commerce invoicing system defines a global baseline tax rate. Inside `main()`, a subsidized local tax rate is defined with the same identifier. Within an inner block, an integer corporate tax rate is declared, again shadowing the identifier. The developer attempts to calculate totals using both local and global tax rates using scope resolution, but encounters compilation and semantic errors.

Buggy Code

```
#include <iostream>

double taxRate = 0.18; // Global Standard Tax Rate (18%)

int main() {
    double itemPrice = 1000.0;
    double taxRate = 0.05; // Local Subsidized Rate (5%)

    std::cout << "Local Rate: " << taxRate << std::endl;

    // Line 11: Developer attempts to use the global tax rate
```

```
    double totalWithGlobalTax = itemPrice + (itemPrice * :taxRate);

    {
        int taxRate = 12; // Nested local variable shadowing

        // Line 16: Developer attempts to access the outer local rate (0.05)
        double totalWithLocalTax = itemPrice + (itemPrice * ::taxRate);
        std::cout << "Inner total: " << totalWithLocalTax << std::endl;
    }

    std::cout << "Final Global Calculation: " << totalWithGlobalTax << std::endl;
    return 0;
}
```

Student Tasks

1. Identify the syntax error on **Line 11**.

2. Explain what value `::taxRate` actually resolves to on **Line 16**. Can the unary scope resolution operator `::` access an enclosing local function scope? Explain C++ scoping rules regarding nested shadowing.

3. Rewrite the code cleanly, resolving the naming ambiguities without compiler errors.

4. Provide the expected console output.

Question 3: Switch-Case Variable Initialization & Illegal Case Types

Topic: Control Structures (`switch-case`), Tokens, Data Types

Problem Statement

A lexical scanner evaluates token types using a `switch` statement. A developer declares and initializes local transformation variables inside individual `case` blocks and uses floating-point values for specific token categories. The compiler rejects the code with multiple fatal errors.

Buggy Code

```
#include <iostream>

int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;

    switch (tokenType) {
        case 1:
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;

        case 2:
            int multiplier = 5; // Error occurs here!
            tokenValue = 20.0f * multiplier;
            // Developer forgot break statement here

        case 3:
```

```
            tokenValue += 100.0f;
            break;

        case 4.5: // Error occurs here!
            std::cout << "Float token detected" << std::endl;
            break;

        default:
            std::cout << "Unknown token" << std::endl;
    }

    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

Student Tasks

1. Explain why initializing `int multiplier = 5;` inside `case 2` causes the compiler to flag an error regarding cross-initialization.

2. State the C++ language rule that causes `case 4.5:` to be rejected. What data types are legally permitted in `switch` expressions and `case` labels?

3. Identify the logical defect that occurs when `tokenType == 2` executes due to missing control statements.

4. Provide the fully corrected and properly formatted code snippet.

Question 4: Reference Variable Initialization & Constant Binding Violations

Topic: Reference Variables, Constants, Implicit Conversions

Problem Statement

A junior programmer attempts to use C++ reference variables as aliases for variables, literals, and type-converted values. The compiler rejects the program due to violations of reference initialization rules.

Buggy Code

```
#include <iostream>

int main() {
    int a = 25;

    // Line 7: Reference declaration without immediate binding
    int &refA;
    refA = a;

    // Line 11: Non-const lvalue reference bound to an rvalue literal
    int &refLiteral = 100;

    // Line 14: Non-const reference bound to an expression requiring conversion
    double pi = 3.14159;
    int &refInt = pi;

    // Line 17: Modification through a const reference
```

```
    const int &constRef = 50;
    constRef = constRef + 10;

    std::cout << a << " " << refA << std::endl;
    return 0;
}
```

Student Tasks

1. Analyze each of the four reference operations (**Lines 7, 11, 14, and 18**) and state the specific C++ rule violated by each.

2. Explain the mechanism of *lifetime extension of temporary objects* when binding to `const` references (`const int&`).

3. Write the corrected code snippet ensuring all references are valid and type-safe.

4. Provide the expected console output.

Question 5: Dynamic Array Allocation & Stray Loop Semicolon

Topic: Memory Operators (`new`, `delete`), Loops, Off-by-One Boundaries

Problem Statement

A telemetry processor dynamically allocates memory on the heap for an array of 5 integer sensor readings. The developer attempts to initialize the array using a loop and compute the sum. However, the program fails to compile or exhibits memory corruption and incorrect calculations.

Buggy Code

```
#include <iostream>

int main() {
    int size = 5;

    // Line 7: Allocating dynamic array
    int *readings = new int[size];

    // Line 10: Loop initialization with a punctuation defect
    for (int i = 0; i < size; ++i);
    {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }
    std::cout << "Total: " << total << std::endl;

    // Line 21: Heap memory deallocation
    delete readings;

    return 0;
}
```

Student Tasks

1. Explain the consequence of the semicolon `;` at the end of the `for` statement on **Line 10**. What does the block `{ readings[i] = ...; }` do, and what scope issue affects `i`?

2. Identify the defect on **Line 21** regarding dynamic memory deallocation. What is the difference between `delete` and `delete[]` in C++?

3. Provide the corrected C++ program.

4. Trace the execution and show the final printed `Total`.

---

4. Section B: Pseudo-Code Writing Questions (5 Problems)

Question 1: Procedural-to-OOP Paradigm Transformation

Topic: OOP Paradigm, Encapsulation, Data Hiding, Abstraction

Scenario

A municipal water authority wants to replace its legacy procedural C scripts with an Object-Oriented design. You are tasked with designing the pseudo-code for a `WaterConsumer` class that encapsulates consumer details, enforces strict data validation on meter readings, and computes tiered bills.

Functional Requirements

1. Attributes (Private):

- `consumerID` (STRING)
- `previousReading` (REAL, cubic meters)
- `currentReading` (REAL, cubic meters)
- `unitsConsumed` (REAL)
- `totalBill` (REAL)

2. Methods (Public):

- `Initialize(id, prev, curr)`: Validates that readings are non-negative and `currentReading >= previousReading`. If invalid, flags an error.
- `CalculateConsumption()`: Determines net units consumed.
- `ComputeBill()`: Computes payable amount using the following tiered tariff:
  - First 100 units: \$1.50 per unit.
  - Next 200 units (101 to 300): \$2.50 per unit.
  - Units above 300: \$4.00 per unit, plus a flat 10% environmental surcharge on the entire bill.
- `DisplayBill()`: Formats and prints the complete invoice.

Student Tasks

1. Write the formal structured pseudo-code defining the `WaterConsumer` class and an executable driver routine `Main`.

2. Construct a dry-run trace table using sample input: `previousReading = 1450.0`, `currentReading = 1820.0`. Show the step-by-step breakdown of units across each tier, surcharge, and final total.

Question 2: Multi-Tier Tax & Precision Cast Engine

Topic: Control Structures (Selection), Data Types, Type Casting

Scenario

An enterprise payroll accounting module needs an algorithm to compute annual personal income tax from gross income and investment deductions, ensuring compliance with statutory deduction caps and precision rounding.

Functional Requirements

1. **Inputs**:
   - `grossAnnualSalary` (REAL)
   - `deduction80C` (REAL, statutory cap: \$150,000)
   - `deductionHealthInsurance` (REAL, statutory cap: \$25,000)
2. **Taxable Income Computation**:
   - Deduct eligible investments from gross income (clamped to zero if deductions exceed gross).
3. **Progressive Tax Brackets**:
   - Up to \$250,000: 0%
   - \$250,001 to \$500,000: 5% on amount over \$250,000
   - \$500,001 to \$1,000,000: 20% on amount over \$500,000
   - Above \$1,000,000: 30% on amount over \$1,000,000
4. **Surcharge**:
   - Add a 4% Health & Education Cess on the total raw tax.
5. **Type Casting Requirement**:
   - Explicitly convert the final tax amount from `REAL` to `INTEGER` (simulating `static_cast<long>(tax + 0.5)` for nearest rupee rounding).

Student Tasks

1. Write the structured pseudo-code for `TaxCalculationEngine`.
2. Provide a dry-run trace table for an employee with `grossAnnualSalary = $1,200,000`, `deduction80C = $180,000`, and `deductionHealthInsurance = $30,000`.

Question 3: Bitwise Mask Validation & Sensor Health Telemetry

Topic: Operators in C++ (Bitwise `&`, `|`, `^`, `~`), Tokens, Constants

Scenario

In an autonomous unmanned aerial vehicle (UAV), an 8-bit unsigned hardware register (`statusRegister`) reports subsystem health flags:

- Bit 0 (0x01): Gyroscope Locked
- Bit 1 (0x02): Accelerometer Calibrated
- Bit 2 (0x04): Barometric Altimeter Online
- Bit 3 (0x08): GPS Satellite Fix Acquired

- Bit 4 (0x10): Low Battery Warning
- Bit 5 (0x20): Over-Temperature Warning
- Bit 6 (0x40): Radio Telemetry Link Active
- Bit 7 (0x80): Fatal Actuator Hardware Failure

Functional Requirements

1. Flight Readiness Conditions:

- **Mandatory**: Bits 0, 1, 2, and 3 must all be `1` (all essential sensors locked).
- **Inhibitors**: Bits 4, 5, and 7 must all be `0` (no warnings or fatal failures).
- Bit 6 (Radio Link) is non-blocking (can be 0 or 1).

2. Logic Rules:

- Must use bitwise masks (`AND`, `OR`, `XOR`, `NOT`) without looping over bit positions.
- If any inhibitor bit is set, emit specific error diagnostics identifying the exact fault bit.
- If mandatory sensors are missing, compute the XOR difference mask to report missing sensor IDs.

Student Tasks

1. Define the required hexadecimal bitmask constants.

2. Write structured pseudo-code for `CheckFlightReadiness(statusRegister)`.

3. Provide a trace for two test bytes: `0x4F` (nominal flight) and `0x3B` (low battery with missing sensor).

Question 4: Optimized Prime Factorization & Divisor Engine

Topic: Control Structures (Iteration: `for`/`while`, Jump: `break`, `continue`, `goto`)

Scenario

Develop an algorithm that accepts a positive integer N, verifies its validity using input validation jumps, determines all proper divisors of N in $O(\sqrt{N})$ time complexity, and classifies N as **Perfect**, **Abundant**, or **Deficient**.

Functional Requirements

1. Input Validation:

- Prompt user for N. If $N \le 0$, display an error message and repeat using a jump/loop construct.
- Handle edge case $N = 1$ explicitly.

2. Divisor Generation:

- Loop candidate divisor d from 2 up to $\lfloor\sqrt{N}\rfloor$.
- Use jump statement `CONTINUE` when d does not divide N evenly.
- When d divides N, accumulate both d and its paired divisor (N / d) (avoiding double-counting when $d = \sqrt{N}$).

3. Classification:

- Sum of proper divisors (including 1):
  - If $\text{Sum} == N$: **Perfect Number** (e.g., 6, 28).
  - If $\text{Sum} > N$: **Abundant Number** (e.g., 12).
  - If $\text{Sum} < N$: **Deficient Number** (e.g., 15).

Student Tasks

1. Write the structured pseudo-code for `DivisorAndPrimeEngine`.

2. Trace the algorithm with $N = 28$ and $N = 12$, showing values of d, (N / d), and `sumOfDivisors` at each step.

Question 5: State-Machine ATM Controller with Sentinel Loops

Topic: Control Structures (`switch-case`, `while`, nested `if-else`), Data Types

Scenario

Design the pseudo-code for an Automated Teller Machine (ATM) transaction manager. The system must authenticate the cardholder via a 4-digit PIN (permitting up to 3 attempts), and then present a menu-driven session loop.

Functional Requirements

1. Authentication Phase:

- Preset PIN = `4821`.
- Allow up to 3 attempts. If incorrect 3 times, lock the card and terminate immediately.

2. Session Loop (Sentinel-controlled):

- Display menu: `1: Balance Inquiry`, `2: Cash Deposit`, `3: Cash Withdrawal`, `4: Exit`.
- **Balance Inquiry**: Display current available balance.
- **Deposit**: Accept positive amount and add to balance.
- **Withdrawal**:
  - Check if amount > 0 and amount $\le$ balance.
  - Validate that withdrawal amount is an exact multiple of 100 (using modulus arithmetic). If not, reject transaction.
- **Exit**: Terminate session gracefully.
- **Default**: Handle invalid option selections.

Student Tasks

1. Write the formal structured pseudo-code for `ATM_SessionController`.

2. Provide a dry-run test trace demonstrating:

- Entering one wrong PIN followed by the correct PIN.
- Attempting to withdraw \$250 (declined due to non-multiple of 100).
- Withdrawing \$300 successfully.

SECTION A – DEBUGGING QUESTIONS

Question 1: Operator Precedence and Unsigned Countdown

- Line 7 – Logic defect: == has higher precedence than bitwise &, so num & 1 == 1 is parsed as num & (1 == 1), effectively num & 1. Compare the masked result explicitly: (num & 1U) == 1U.
- Line 14 – Runtime/control-flow defect: unsigned char cannot represent a negative value. Decrementing zero wraps to its maximum value, so i >= 0 remains true. Use a signed counter for a countdown.

```
#include <iostream>
int main() {
    unsigned char num = 15;
    if (((num & 1U) == 1U) && (num > 10U))
        std::cout << "Valid Odd Number: " << static_cast<int>(num) << std::endl;
    else
        std::cout << "Condition Failed for: " << static_cast<int>(num) << std::endl;

    for (int i = 3; i >= 0; --i) std::cout << i << " ";
    std::cout << std::endl;
    return 0;
}
```

Dry run: num = 15; 15 & 1 = 1 (odd); 15 > 10; both conditions are true. The signed loop prints 3, 2, 1, 0 and then terminates.

```
Valid Odd Number: 15
3 2 1 0
```

Question 2: Scope Resolution and Shadowing

- Line 11 – Compilation error: the global scope operator is ::, not :. Correct expression: ::taxRate.
- Line 16 – Scope defect: ::taxRate always names the global variable. It cannot access an enclosing local variable in main(). Give the outer local rate a distinct name and refer to it directly.

```
#include <iostream>
double taxRate = 0.18;

int main() {
    double itemPrice = 1000.0;
    double subsidizedRate = 0.05;
    std::cout << "Local Rate: " << subsidizedRate << std::endl;

    double totalWithGlobalTax = itemPrice + itemPrice * ::taxRate;
    {
        int corporateRate = 12;
        double totalWithLocalTax = itemPrice + itemPrice * subsidizedRate;
        std::cout << "Corporate Rate: " << corporateRate << std::endl;
        std::cout << "Inner total: " << totalWithLocalTax << std::endl;
    }
    std::cout << "Final Global Calculation: " << totalWithGlobalTax << std::endl;
    return 0;
}
```

Dry run: local total = 1000 + 1000 × 0.05 = 1050; global total = 1000 + 1000 × 0.18 = 1180.

```
Local Rate: 0.05
Corporate Rate: 12
Inner total: 1050
Final Global Calculation: 1180
```

Question 3: Switch-Case Initialization and Illegal Case Types

- Cross-initialization: a switch may jump directly to a case label, bypassing an initialization in the same switch scope. Enclose declarations in braces within each case.
- case 4.5 is invalid: switch expressions and case labels use integral or enumeration types; floating-point case labels are not allowed. Use an integer token code.
- Missing break in case 2 causes fall-through into case 3. For tokenType 2, the value would become 100 + 100 = 200. Add break.

```
#include <iostream>
int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;
    switch (tokenType) {
        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }
        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;
        }
        case 3: {
            tokenValue += 100.0f;
            break;
        }
        case 4: {
            std::cout << "Integer token detected" << std::endl;
            break;
        }
        default: {
            std::cout << "Unknown token" << std::endl;
            break;
        }
    }
    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

Dry run: tokenType = 2; multiplier = 5; tokenValue = 20 × 5 = 100; break exits the switch.

```
Calculated Value: 100
```

Question 4: Reference Initialization and Const Binding

- Line 7 – Compilation error: a reference must be initialized at declaration; int& refA must bind immediately to an int lvalue.
- Line 11 – Compilation error: a non-const lvalue reference cannot bind to the temporary literal 100. A const int& can bind to it.
- Line 14 – Compilation error: int& cannot bind to the temporary int produced by converting double pi. A const int& can bind to that converted temporary.
- Line 18 – Compilation error: constRef is read-only; assignment through it is forbidden.

A temporary directly bound to a const lvalue reference has its lifetime extended to the lifetime of that reference within the applicable scope.

```
#include <iostream>
int main() {
    int a = 25;
    int& refA = a;
    const int& refLiteral = 100;
    double pi = 3.14159;
    const int& refInt = static_cast<int>(pi);
    const int& constRef = 50;
    // constRef = constRef + 10; // Invalid: const reference is read-only

    std::cout << a << " " << refA << std::endl;
    std::cout << refLiteral << " " << refInt << " " << constRef << std::endl;
    return 0;
}
```

Dry run: refA aliases a (25); refLiteral is 100; static_cast<int>(3.14159) is 3; constRef is 50.

```
25 25
100 3 50
```

Question 5: Dynamic Array and Stray Loop Semicolon

- Line 10 – Logic and compilation defect: the semicolon makes the for loop body empty. The following block is outside the loop, and i is out of scope there.
- Line 21 – Undefined behavior: memory allocated using new[] must be released using delete[].

```
#include <iostream>
int main() {
    const int size = 5;
    int* readings = new int[size];

    for (int i = 0; i < size; ++i) {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) total += readings[i];

    std::cout << "Total: " << total << std::endl;
    delete[] readings;
    return 0;
}
```

i	Reading	Running total
0	10	10
1	20	30
2	30	60
3	40	100
4	50	150

```
Total: 150
```

new[] allocates an array; delete[] releases the entire array and invokes the appropriate destructors. A standard container such as vector is generally safer in production code.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Pseudo-code uses uppercase control keywords, explicit variable types, and the CLASS / PRIVATE / PUBLIC / METHOD / ENDCLASS structure requested in the question bank.

Question 1: Procedural-to-OOP WaterConsumer

- Encapsulation: consumer details and billing values are private; public methods validate readings, calculate consumption, compute the tiered bill, and display the invoice.
- Validation rejects negative readings and current readings below previous readings. The surcharge is applied only when consumption is above 300 units.

```
CLASS WaterConsumer
    PRIVATE:
        consumerID: STRING
        previousReading: REAL
        currentReading: REAL
        unitsConsumed: REAL
        totalBill: REAL
        isValid: BOOLEAN

    PUBLIC:
        CONSTRUCTOR Initialize(id: STRING, prev: REAL, curr: REAL)
            consumerID <- id
            isValid <- FALSE
            unitsConsumed <- 0
            totalBill <- 0
            IF prev < 0 OR curr < 0 OR curr < prev THEN
                OUTPUT "ERROR: Invalid meter readings"
                RETURN
            ENDIF
            previousReading <- prev
            currentReading <- curr
            isValid <- TRUE
        END

        METHOD CalculateConsumption()
            IF isValid = FALSE THEN RETURN ENDIF
            unitsConsumed <- currentReading - previousReading
        END

        METHOD ComputeBill()
            DECLARE baseBill: REAL
            DECLARE remaining: REAL
            DECLARE surcharge: REAL
            IF isValid = FALSE THEN RETURN ENDIF
            CALL CalculateConsumption()
            remaining <- unitsConsumed
            baseBill <- 0

            IF remaining <= 100 THEN
                baseBill <- remaining * 1.50
            ELSE
                baseBill <- 100 * 1.50
                remaining <- remaining - 100
                IF remaining <= 200 THEN
```

```
                baseBill <- baseBill + remaining * 2.50
            ELSE
                baseBill <- baseBill + 200 * 2.50
                remaining <- remaining - 200
                baseBill <- baseBill + remaining * 4.00
            ENDIF
        ENDIF

        surcharge <- 0
        IF unitsConsumed > 300 THEN
            surcharge <- baseBill * 0.10
        ENDIF
        totalBill <- baseBill + surcharge
    END

    METHOD DisplayBill()
        IF isValid = FALSE THEN
            OUTPUT "Invoice unavailable"
            RETURN
        ENDIF
        OUTPUT "Consumer ID: ", consumerID
        OUTPUT "Previous Reading: ", previousReading
        OUTPUT "Current Reading: ", currentReading
        OUTPUT "Units Consumed: ", unitsConsumed
        OUTPUT "Total Bill: ", totalBill
    END
ENDCLASS

BEGIN
    DECLARE consumer: WaterConsumer
    CALL consumer.Initialize("WC101", 1450.0, 1820.0)
    CALL consumer.CalculateConsumption()
    CALL consumer.ComputeBill()
    CALL consumer.DisplayBill()
END
```

Stage	Calculation	Value
Consumption	1820 − 1450	370 units
First 100 units	100 × 1.50	150.00
Next 200 units	200 × 2.50	500.00
Above 300 units	70 × 4.00	280.00
Base bill	150 + 500 + 280	930.00
Environmental surcharge	930 × 10%	93.00
Final bill	930 + 93	1,023.00

Final invoice total for consumer WC101 = 1,023.00.

Question 2: Multi-Tier Tax and Precision Cast Engine

- Cap deductions at 150,000 and 25,000 respectively, clamp taxable income to zero, apply each marginal tax rate only to its bracket, add 4% cess, and round to the nearest whole rupee.

```
CLASS TaxCalculationEngine
    PRIVATE:
        grossAnnualSalary: REAL
        eligible80C: REAL
```

```
        eligibleHealth: REAL
        taxableIncome: REAL
        rawTax: REAL
        cess: REAL
        finalTax: INTEGER

    PUBLIC:
        METHOD CalculateTax(gross: REAL, d80: REAL, health: REAL)
            grossAnnualSalary <- MAX(0, gross)
            eligible80C <- MIN(MAX(0, d80), 150000)
            eligibleHealth <- MIN(MAX(0, health), 25000)

            taxableIncome <- grossAnnualSalary - eligible80C - eligibleHealth
            IF taxableIncome < 0 THEN taxableIncome <- 0 ENDIF

            rawTax <- 0
            IF taxableIncome > 1000000 THEN
                rawTax <- rawTax + 500000 * 0.20
                rawTax <- rawTax + 250000 * 0.05
                rawTax <- rawTax + (taxableIncome - 1000000) * 0.30
            ELSE IF taxableIncome > 500000 THEN
                rawTax <- rawTax + 250000 * 0.05
                rawTax <- rawTax + (taxableIncome - 500000) * 0.20
            ELSE IF taxableIncome > 250000 THEN
                rawTax <- (taxableIncome - 250000) * 0.05
            ENDIF

            cess <- rawTax * 0.04
            finalTax <- INTEGER(rawTax + cess + 0.5)

            OUTPUT "Eligible 80C: ", eligible80C
            OUTPUT "Eligible Health: ", eligibleHealth
            OUTPUT "Taxable Income: ", taxableIncome
            OUTPUT "Raw Tax: ", rawTax
            OUTPUT "Cess: ", cess
            OUTPUT "Final Tax: ", finalTax
        END
ENDCLASS

BEGIN
    DECLARE engine: TaxCalculationEngine
    CALL engine.CalculateTax(1200000, 180000, 30000)
END
```

Item	Calculation	Value
Gross salary	Given	1,200,000
Eligible 80C	MIN(180,000, 150,000)	150,000
Eligible health	MIN(30,000, 25,000)	25,000
Taxable income	1,200,000 − 150,000 − 25,000	1,025,000
0% bracket	250,000 × 0%	0
5% bracket	250,000 × 5%	12,500
20% bracket	500,000 × 20%	100,000
30% bracket	25,000 × 30%	7,500
Raw tax	12,500 + 100,000 + 7,500	120,000
4% cess	120,000 × 0.04	4,800
Final tax	INTEGER(124,800 + 0.5)	124,800

Final tax payable = 124,800.

Question 3: Bitwise Mask Validation and Sensor Health

Required constants: MANDATORY_MASK = 0x0F; INHIBITOR_MASK = 0xB0; LOW_BATTERY_MASK = 0x10; OVER_TEMP_MASK = 0x20; FATAL_FAILURE_MASK = 0x80.

Bit	Mask	Meaning
0	0x01	Gyroscope locked – mandatory
1	0x02	Accelerometer calibrated – mandatory
2	0x04	Barometric altimeter online – mandatory
3	0x08	GPS fix acquired – mandatory
4	0x10	Low battery – inhibitor
5	0x20	Over-temperature – inhibitor
6	0x40	Radio link – non-blocking
7	0x80	Fatal actuator failure – inhibitor

```
METHOD CheckFlightReadiness(statusRegister: BYTE)
    DECLARE missingMask: BYTE
    DECLARE ready: BOOLEAN
    ready <- TRUE

    IF (statusRegister AND 0x10) != 0 THEN
        OUTPUT "ERROR: Low battery warning"
        ready <- FALSE
    ENDIF
    IF (statusRegister AND 0x20) != 0 THEN
        OUTPUT "ERROR: Over-temperature warning"
        ready <- FALSE
    ENDIF
    IF (statusRegister AND 0x80) != 0 THEN
        OUTPUT "ERROR: Fatal actuator hardware failure"
        ready <- FALSE
    ENDIF

    missingMask <- 0x0F XOR (statusRegister AND 0x0F)
    IF missingMask != 0 THEN
        OUTPUT "Missing mandatory sensor mask: ", HEX(missingMask)
        IF (missingMask AND 0x01) != 0 THEN OUTPUT "Gyroscope missing" ENDIF
        IF (missingMask AND 0x02) != 0 THEN OUTPUT "Accelerometer missing" ENDIF
        IF (missingMask AND 0x04) != 0 THEN OUTPUT "Barometric altimeter missing" ENDIF
        IF (missingMask AND 0x08) != 0 THEN OUTPUT "GPS fix missing" ENDIF
        ready <- FALSE
    ENDIF

    IF ready = TRUE THEN
        OUTPUT "FLIGHT READY"
    ELSE
        OUTPUT "FLIGHT NOT READY"
    ENDIF
END
```

Input	Mandatory bits / XOR mask	Inhibitors	Result
0x4F	0x4F AND 0x0F = 0x0F; missing = 0x00	0x4F AND 0xB0 = 0x40 (radio only)	Flight ready
0x3B	0x3B AND 0x0F = 0x0B; missing = 0x0F XOR 0x0B = 0x04	Bit 4 and bit 5 are set: low battery and over-temperature	Flight not ready; barometric altimeter missing

Note: Although the scenario describes 0x3B as a low-battery case, the byte also sets bit 5 (0x20). Therefore, the required diagnostics must report over-temperature as well as low battery.

Question 4: Optimized Divisor and Classification Engine

- Validate N > 0 by repeating input. Handle N = 1 separately. For N > 1, initialize the proper-divisor sum to 1.
- Test divisors from 2 through floor(√N). If d does not divide N, use CONTINUE. Otherwise add d and its paired divisor N/d; avoid adding the same square-root divisor twice.

```
CLASS DivisorAndPrimeEngine
    PUBLIC:
        METHOD Analyze()
            DECLARE N: INTEGER
            DECLARE d: INTEGER
            DECLARE pairedDivisor: INTEGER
            DECLARE sumOfDivisors: INTEGER

            INPUT N
            WHILE N <= 0 DO
                OUTPUT "Invalid input. Enter a positive integer."
                INPUT N
            ENDWHILE

            IF N = 1 THEN
                OUTPUT "Sum of proper divisors: 0"
                OUTPUT "Deficient Number"
                RETURN
            ENDIF

            sumOfDivisors <- 1
            d <- 2
            WHILE d * d <= N DO
                IF N MOD d != 0 THEN
                    d <- d + 1
                    CONTINUE
                ENDIF

                pairedDivisor <- N / d
                sumOfDivisors <- sumOfDivisors + d
                IF pairedDivisor != d THEN
                    sumOfDivisors <- sumOfDivisors + pairedDivisor
                ENDIF
                d <- d + 1
            ENDWHILE

            OUTPUT "Sum of proper divisors: ", sumOfDivisors
```

```
            IF sumOfDivisors = N THEN
                OUTPUT "Perfect Number"
            ELSE IF sumOfDivisors > N THEN
                OUTPUT "Abundant Number"
            ELSE
                OUTPUT "Deficient Number"
            ENDIF
        END
ENDCLASS

BEGIN
    DECLARE engine: DivisorAndPrimeEngine
    CALL engine.Analyze()
END
```

Time complexity: O(√N), because candidate divisors are tested only up to the square root. Auxiliary space complexity: O(1).

Trace for N = 28

d	28 MOD d	Paired divisor	Running sum
Initial	—	—	1
2	0	14	1 + 2 + 14 = 17
3	1	—	17 (CONTINUE)
4	0	7	17 + 4 + 7 = 28
5	1	—	28 (CONTINUE)
Stop	6 × 6 > 28	—	28

28 is a Perfect Number because the sum of its proper divisors is 28.

Trace for N = 12

d	12 MOD d	Paired divisor	Running sum
Initial	—	—	1
2	0	6	1 + 2 + 6 = 9
3	0	4	9 + 3 + 4 = 16
Stop	4 × 4 > 12	—	16

12 is an Abundant Number because the sum of its proper divisors is 16, which is greater than 12.

Question 5: State-Machine ATM Controller

- Authentication allows at most three PIN attempts. A correct PIN enables the session; three wrong PINs lock the card.
- The session loop repeats until option 4. Withdrawals must be positive, within the balance, and exact multiples of 100.

```
CLASS ATM_SessionController
    PRIVATE:
        presetPIN: INTEGER <- 4821
        balance: REAL
        attempts: INTEGER
        authenticated: BOOLEAN
        sessionActive: BOOLEAN

    PUBLIC:
```

```
CONSTRUCTOR Initialize(openingBalance: REAL)
    balance <- openingBalance
    attempts <- 0
    authenticated <- FALSE
    sessionActive <- FALSE
END

METHOD Authenticate()
    DECLARE enteredPIN: INTEGER
    WHILE attempts < 3 AND authenticated = FALSE DO
        INPUT enteredPIN
        IF enteredPIN = presetPIN THEN
            authenticated <- TRUE
            OUTPUT "Authentication successful"
        ELSE
            attempts <- attempts + 1
            OUTPUT "Incorrect PIN"
        ENDIF
    ENDWHILE

    IF authenticated = FALSE THEN
        OUTPUT "Card locked. Too many incorrect attempts."
        RETURN
    ENDIF
    sessionActive <- TRUE
END

METHOD RunSession()
    DECLARE choice: INTEGER
    DECLARE amount: REAL
    WHILE sessionActive = TRUE DO
        OUTPUT "1. Balance Inquiry"
        OUTPUT "2. Cash Deposit"
        OUTPUT "3. Cash Withdrawal"
        OUTPUT "4. Exit"
        INPUT choice

        SWITCH choice
            CASE 1:
                OUTPUT "Available balance: ", balance
                BREAK
            CASE 2:
                INPUT amount
                IF amount > 0 THEN
                    balance <- balance + amount
                    OUTPUT "Deposit successful"
                ELSE
                    OUTPUT "Invalid deposit amount"
                ENDIF
                BREAK
            CASE 3:
                INPUT amount
                IF amount <= 0 THEN
                    OUTPUT "Invalid withdrawal amount"
                ELSE IF amount > balance THEN
                    OUTPUT "Insufficient balance"
```

```
                    ELSE IF amount MOD 100 != 0 THEN
                        OUTPUT "Declined: amount must be a multiple of 100"
                    ELSE
                        balance <- balance - amount
                        OUTPUT "Withdrawal successful"
                        OUTPUT "Remaining balance: ", balance
                    ENDIF
                    BREAK
                CASE 4:
                    sessionActive <- FALSE
                    OUTPUT "Thank you. Session ended."
                    BREAK
                DEFAULT:
                    OUTPUT "Invalid menu option"
                    BREAK
            ENDSWITCH
        ENDWHILE
    END
ENDCLASS

BEGIN
    DECLARE atm: ATM_SessionController
    CALL atm.Initialize(1000.00)
    CALL atm.Authenticate()
    IF atm.authenticated = TRUE THEN
        CALL atm.RunSession()
    ENDIF
END
```

Trace assumption: the assignment does not provide an opening balance, so this demonstration starts with 1,000.00.

Step	Input / action	System response	Balance
1	Wrong PIN: 1111	Incorrect PIN; one attempt used.	1,000.00
2	Correct PIN: 4821	Authentication successful; menu shown.	1,000.00
3	Choose 3; amount 250	Declined: not a multiple of 100; balance unchanged.	1,000.00
4	Choose 3; amount 300	Withdrawal successful.	700.00
5	Choose 4	Thank you. Session ended.	700.00

```
Incorrect PIN
Authentication successful
Declined: amount must be a multiple of 100
Withdrawal successful
Remaining balance: 700
Thank you. Session ended.
```